

Project Assignment

Advanced Programming

2025-2026

1 Introduction

This year's project consists of designing and implementing an interactive game inspired by the classic Bomberman battle mode¹, in C++ and using the SFML² graphics library. The main goal of this project is to demonstrate that you are able to create a well-designed architecture, fully utilize advanced C++ features and provide high-quality code that implements the requirements. Of course, it's great if you add creative extra features or fancy graphics and animations, but make sure the basics work well first, and you have a good, extendable codebase to work with. You can play the game here and use this as a reference for your project. Note that this version of the game is not exactly the same as the project described here!

2 Gameplay

Bomberman is a game in which the Player competes against three computer-controlled characters to be the last man standing. They do this by using bombs to break blocks, gather power-ups and trap the other players!

The detailed gameplay requirements are divided into three sections: core functionalities, visuals and aesthetics, and optional extensions for bonus points.



Figure 1: Different versions of Bomberman Battle Mode.

2.1 Core Functionalities

Game startup & Initialization: When launching the game, the user is presented with an interface displaying the scoreboard, containing the top five scores. Below, a “Play” button is displayed. Clicking this button clears the screen and starts the game.

Choose a representation for the arena. How will you represent walls, the Player, the enemies, power-ups and other elements in the arena? For the arena, you can replicate the layout shown in figure 1, consisting of alternating indestructible blocks, filling in the remaining blocks with destructible blocks with a low chance of spawning air instead. It is not important if the aesthetics of your game do not exactly match those from the figure.

Player Controls & Interactions with Objects: Implement controls for moving through the arena. Ensure the Player sprite responds to player input (e.g., arrow keys). Movement

¹https://bomberman.fandom.com/wiki/Battle_Game

²<https://www.sfml-dev.org/>

should be continuous and not discrete. Implement collision detection to prevent the Player from moving through walls. The player should spawn in the top left corner.

Bombs and Explosions: When pressing a key (e.g. spacebar) the Player can place a bomb at their current location. After moving out of the bomb, the player can no longer go through it. The bomb explodes after a brief amount of time (e.g. 2 seconds) in a cross-shaped pattern destroying any destructible blocks in this radius. Explosions cannot go through indestructible blocks, and can only go through one destructible block at a time in each direction. Explosions can trigger other bombs in their radius, causing a chain reaction of explosions. Any entities hit by an explosion immediately die and lose the game! Explosions also destroy any power-ups within range.

At the start of the game, the Player and each of the bots only have one bomb available with a radius of one in each direction. After a bomb explodes, it can be placed once again.

Power-ups: When breaking destructible blocks, there is a chance (e.g. 25%) it will spawn a power-up. When a character walks over one of these power-ups they gain a permanent stat boost. For this project, you are expected to implement the following power-ups:

- **Fire:** Increases bomb blast radius by one in each direction.
- **Extra Bomb:** Increases the amount of bombs a player can place at any given time.
- **Skates:** Increases the character's movement speed.

Enemies and AI: Create three computer-controlled enemies in each of the three remaining corners. You are free to make them each behave slightly differently, though their AI should at least do the following:

- Basic survival instincts; if a bomb is going to blow them up, they should attempt to run to safety.
- If any power-ups are in their range, they should try to pick them up.
- If any destructible walls are in their range, they should regularly place bombs next to them to increase their playfield.
- If any enemies are nearby or no breakable walls remain, they should try to kill them with their bombs.
- Bots should be able to pick up and understand power-ups. If they have multiple bombs, they should be able to place them all. If they have a bigger bomb radius, they should understand that they need to escape further away.

Scoring: Display a game score based at least the following factors (though you are allowed to add more):

- Time the Player is alive
- Blocks broken by the Player's bombs
- Power-ups picked up by the Player
- Enemies killed by the Player
- Big point bonus for winning, and penalty for losing

Maintain a scoreboard with the top five scores and ensure it's visible at game launch.

2.2 Visuals and Aesthetics

For this game, you can use any sprites you would like, as long as the game mechanics are all still clearly present. You can also use one of the many Bomberman sprite sheets available online, such as the ones in figure 2. Note that this figure provides more sprites than necessary for the project. The image, as well as other potential sprite sheets, can be found [here](#). Implement animations for the Player and other enemies: a walking animation should be visible for each of the four possible directions. Upon death, a death animation should play. You can optionally add a victory animation as well. An animation should play when each bomb ticks down. Each explosion should also have a fast animation in which it slowly grows and fades away in each of the affected tiles.

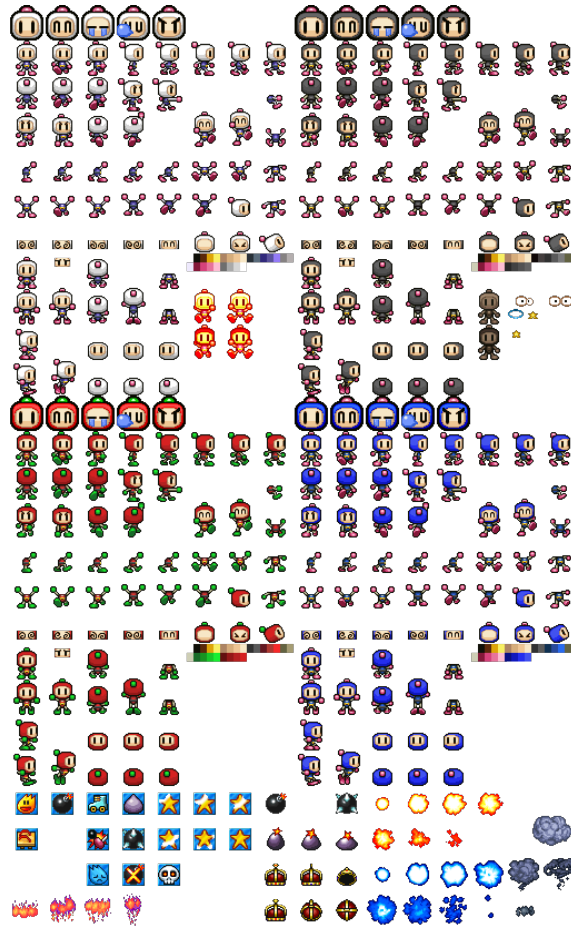


Figure 2: Example sprites for the game. Not all figures should be used.

2.3 Project Extensions

As discussed later in section 4.2, it will be possible to earn up to 10% bonus points by adding extra features to extend your game. Here are some ideas you could consider, or you can try to come up some creative ideas of your own!

- Sounds and Music: Add sounds for events like explosions, death, victory and background music to make the game more enjoyable.
- Smarter bots: Make the bots smarter using different strategies, or give each bot their own personality! One bot could try to avoid players and get as many power-ups as possible. Another might try to trap players using multiple bombs.
- Extra power-ups: Add other power-ups with new features. The original games have explored many different types of power-ups, such as a kick which allows you to kick a bomb away from you or a freeze which slows all the enemies down. Feel free to make up your own power-ups too!

In addition to functional extensions, you can also receive bonus points for technical extensions. Some suggestions include the use of:

- Different Build System: Try using meson instead of CMake to build your project. (Meson is supposedly faster than CMake because of its default use of ninja.)
- Multi-threading: perform more complex computations on multiple threads to make your game run faster. Potentials for this could be your collision detections, more complex enemy

AI with pathfinding, or splitting your logic updates and rendering steps into different threads. But be careful that creating and destroying threads also causes overhead, so it's not always worth it to start a new thread for simple computations.

- Generic programming: there are quite a few places where you could apply templates to make the overall design of your project even more generic. For example, there's a fancy implementation of the Observer pattern where, through the use of templates instead of inheritance, you can retain the type of the concrete subject you're observing. Or you could simplify the interface of your Abstract Factory by using a single create method that is templated on the type of Entity you want to create.
- Design patterns: include the correct use of additional design patterns, other than the ones listed in section 3.1.1, to elegantly solve problems you encounter. The Command pattern could for example come in handy to resolve what needs to happen when colliding with different types of Entities. Or you could use a Visitor when looping over a list of generic Entities in your World to resolve them to their concrete type.

3 Technical Requirements

3.1 Code Design

An important part of this project is creating a flexible design of the game entities and their interactions, as well as the correct use of design patterns. You'll need to design a class structure for the different game entities that facilitates this and keep in mind that your game should be easily extendable. An essential aspect of this design is that there needs to be a clear separation between game logic and representation. Classes that contain game logic should not contain any code related to representation or the other way around. By having this clear separation, you could easily make an alternative representation using a different graphics library, without needing to change any code related to the game logic. To further facilitate this, you'll have to encapsulate the game logic into a standalone (static or dynamic) library using CMake³ and link your representation code with this library in order to generate the final binary. In theory, you should be able to compile this logic library without having SFML installed. A possible (incomplete) hierarchy could for example look like the following:

You are free in how you design this class structure, but the following key classes need to be present, as well as the use of the design patterns discussed in section 3.1.1:

Game: As part of the game representation, this class is responsible for setting up anything that is not related to the core logic of the game. Examples of this include creating the SFML window, running the main game loop and setting-up the StateManager (see section 3.1.1). Other responsibilities may be delegated to the StateManager or concrete States, such as instantiating a concrete factory and processing user interaction. Note that if the user is currently playing, the game representation can translate specific key presses to actions in the World, such as *move left*, *move right*, *move up* or *move down*. But it should not be responsible for how these actions influence the actual game logic.

Stopwatch: This class keeps the difference in time between the current update step (tick) and the previous one (*deltaTime*). Any time your entities move during an update step, the distance should be multiplied with this *deltaTime* value. This is used to ensure that the game logic runs at the same speed, regardless of the speed of the device it is running on. You will also need this value in your representation code to make sure that animations last for a fixed amount of time before switching to the next sprite. You're not allowed to use busy waiting to slow down devices that are running too quickly; the frame rate needs to be dynamic. The only exception to this is that you can cap the frame rate at a certain maximum value (for example 60 FPS), to match the maximum refresh rate of your display. To implement this class, you must use C++ functionality, not the SFML Clock class.

³<https://cmake.org/>

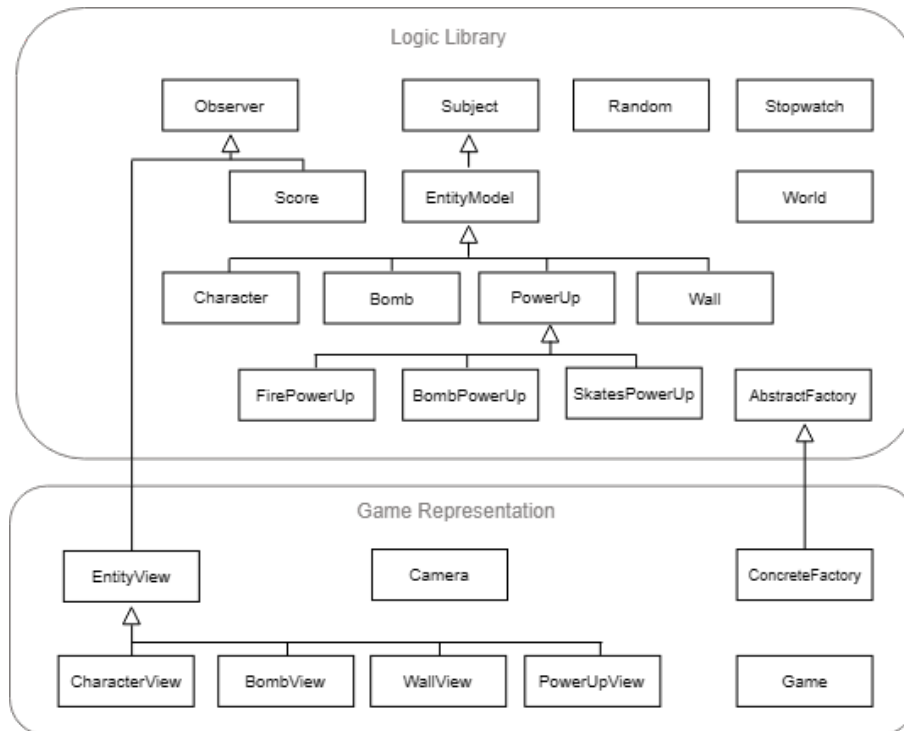


Figure 3: Example class hierarchy with clear separation between logic and representation

World: All entities are stored in the World, which is responsible for orchestrating the overall game logic and the interactions between the entities it contains, such as the creation and destruction of entities and collision detection between them. You can detect these collisions using basic intersecting rectangles, but don't use SFML utilities here, since this class is part of the logic library.

Camera: The Camera class models how the positions of your entities are projected to pixel values on your window. The positions of entities in a World should be modelled using a normalized coordinate system, with the World width and height bounded by $[-1, 1]$. This ensures that the logic of your game is not dependent on the resolution of the display it's running on. Theoretically, it could even be played through a completely text-based interface. You can also use this projection to compute the proper width and height of your sprites in the representation part of your codebase. This functionality again needs to be implemented manually, without relying on SFML utilities.

Score: This class is responsible for computing the current score, by interacting with the Stopwatch class and through events it receives through the Observer pattern (discussed in the next section). Each event should result in a different modifier applied to the current score. It also manages the high scores, so they can be displayed on the starting screen, saving them to a file to make sure it is maintained through different runs of your game.

Random: This class is used to generate all the random numbers you'll need throughout the game, mainly in the AI for your Ghosts. Make sure to use proper C++ functionality for this, through a pseudo-random number generator such as a Mersenne Twister⁴, instead of the legacy rand/srand from C. This generator should be stored as a data member to ensure you receive a new value from the same generator, instead of creating a new generator for each value, increasing randomness significantly.

⁴https://en.cppreference.com/w/cpp/numeric/random/uniform_real_distribution

3.1.1 Design Patterns

You will need to incorporate the following design patterns in your design:

Model-View-Controller (MVC): This pattern is used in order to clearly model the separation between game-state, graphical representation and the logic of the game. In this pattern, the World can be seen as an *Entity Controller* that manages the interactions between entities, or you could have multiple controllers in your design to further delegate responsibilities. Your *Model* is then responsible for holding any data related to a certain entity and to provide some methods that manipulate this data, while the corresponding *View* is responsible for what this entity looks like and for drawing this to the screen.

Observer: You will need to use the Observer pattern for two purposes. Firstly, for updating the *View* when the *Model* state changes. By attaching the *View* observers to the *Model* subjects directly when they are created in your concrete factory, you can separate the logic from the SFML representation completely transparently. A *View* can then receive an event for every update step (tick) to make sure the *Model* is drawn on the window. To make this easier, you could hold a (smart) pointer to your window object in your *View*. It can also receive events when the Player moves in a different direction or when they die, such that the correct animation can be displayed. These same generic events are at the same time used by the Score class (also an Observer) for updating the current score when a block is destroyed, power-up is picked up, when the Player wins or loses.

Abstract Factory: This pattern is used to provide an easy interface which the World can use to create new Entities, without it needing to be aware of how to create instances of SFML-specific *View* classes. The logic library defines a simple abstract factory interface, which is adhered to by a concrete implementation in the representation code. Finally, the Game class provides a pointer to this concrete factory to the World, which can then use it to produce Entities that already have the correct *View* attached.

Singleton: The Stopwatch and Random helper classes needs to be implemented using the Singleton pattern. This ensures that only one instance will be present during the execution of your program, and they are easily accessible to all classes that will need to use them. You could additionally opt to encapsulate your *sf::RenderWindow* in a singleton instead of storing a reference or pointer to it as member in the *View* classes.

3.2 Code Quality

Below you can find a list of things that need to be present in your code to improve its quality:

- Write small **tests** for your game logic. As you have not seen unit testing frameworks yet, you are free to implement these tests in whatever way you prefer.
- Create **diagrams** of your class structure beforehand to plan out your architecture, like the one in figure 3.
- Use **namespaces** to clearly divide modular sections of your code.
- Include **exception handling** to catch and deal with possible errors, such as the absence of required files.
- Proper use of the **static**, **const** and **override** keywords where they can be applied.
- Make sure to avoid memory leaks by explicitly creating **virtual destructors** where necessary.
- Always explicitly initialize **primitive types**. (Hint: check these last two with *valgrind*.)
- Avoid unnecessarily copying objects where they can be passed as a reference or pointer.
- Refrain from relying on **dynamic casts**! This usually means your design is lacking proper polymorphism.

- Avoid duplicate code, solve this by using better **polymorphism** or **templates**.
- Use **clang-format** with *this configuration* to format your code.
- Write proper **code comments** and **API documentation**.
- Use of **smart pointers** throughout the whole project is obligated. This is used to test your insight on where to use unique, shared or weak pointers, depending on the type of ownership. No raw pointers are allowed, except in certain design patterns where the use of smart pointers is prohibitive. But if you need them, please discuss this with us and provide your reasoning first. Passing objects by reference is also perfectly fine, this does not necessarily always need to happen through the use of pointers.
- Keep the design principles in mind. Aim for low coupling and high cohesion in your code.

4 Practical Information

4.1 Additional Resources

Below you can find some useful extra resources to help you get started on your project:

- The *SFML Game Development* book: highly recommended for getting started on working with SFML and some general game concepts, such as how a main game loop works. There's also *SFML Game Development By Example*, which is a similar book that explains the same concepts by guiding you through the process of creating snake clone. The pdfs for both can easily be found online, but let me know if you have trouble finding it.
- *Head First Design Patterns* book: contains an explanation for all the design patterns mentioned in this assignment. Again, the pdf can easily be found online, but there are also more than enough other free resources available that explain these patterns well.
- *The C++ Programming Language* book: contains everything you may need to know about how C++ works to complete this assignment and much, much more.

4.2 Grading

The grading of your project will be divided as follows:

- **20% Core Functionality:** This includes working code, project documentation, and the inclusion of the required design patterns. Your project documentation should include an overview of your design choices and the diagrams of your class structure. Avoid repeating what is already explained in this assignment.
- **40% Code Quality:** This includes the tests you wrote, the diagrams of your class structure, and your application of good design practices.
- **40% Defense:** This will be an actual defense (no longer a submitted video). You will be asked questions about the code you wrote, your design choices, your process and workflow, and how you utilized AI during development.
- **10% Bonus:** You can earn these by implementing creative extra gameplay mechanics, making the game look and feel extra fancy, or making correct use of additional design patterns (see section 2.3). These extra points are simply added to the grade of your project, so your total becomes $\min(20 + 40 + 40 + 10, 100)\%$ in case all parts of your project are perfect. If you added any of these extra features to your project, make sure to also document them thoroughly in your report.

4.3 Submitting

Take the following things into account before submitting your code:

- Your code should **compile and run successfully on the reference platform at the university computer labs**:

Ubuntu: 24.04.3, **SFML:** 2.6.1, **CMake:** 3.28.3, **G++:** 13.2.0, **Clang:** 18.0.0

- This project has to be completed **individually**; plagiarism will not be tolerated. You can of course freely discuss design decisions or implementation problems with GPT tools or other students. The use of AI is allowed and expected, but you must put an effort into understanding your code and the design principles you use. Because of this, your defense will carry significant weight and will test your comprehension of any AI-assisted code.
- Create a GitHub repo and push your project to it (make sure to mention the link to the repo in the report!). Make sure to include your name and student number in the *README.md* file. Frequently commit code increments and set up CI to automatically test whether the project still compiles successfully. The final commit of your project must show a **successful build on a CI platform that is linked to your GitHub repository**. We recommend you use CircleCI⁵ for this, since their free plan allows for considerably more than enough weekly builds for this project and it's easy to set up. You are also allowed to use an alternative CI platform, but make sure that we can see the build results for each commit on GitHub and the build configuration files (e.g. *.circleci/config.yml*).
- If you have any questions, don't hesitate to contact joshi.dhima@uantwerpen.be.
- The project deadline will be **August 22nd**.
- The final project needs to be submitted on **Blackboard** and on **GitHub**.

Good Luck!

⁵<https://circleci.com/>